

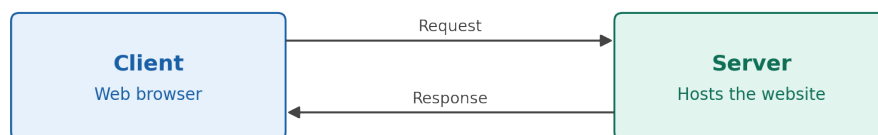
Assignment 1

Web Development Fundamentals

1. Frontend vs Backend vs Full-Stack Development

- **Frontend development** is everything the user sees and interacts with directly in the browser — layout, styling, buttons, forms, animations. Built with HTML, CSS, and JavaScript (often frameworks like React or Vue). Example: the product pages, cart button, and checkout screen on Amazon.
- **Backend development** is the server-side logic that powers the app — handling requests, running business logic, talking to databases, managing authentication. Built with languages like Node.js, Python, Java, PHP. Example: the system on Amazon that checks stock, calculates prices, and processes payment.
- **Full-stack development** means working on both sides — building the interface and the server/database logic behind it. Example: a solo developer building a personal blog site from scratch, coding both the pages people see and the server that stores and serves the posts.

2. Client-Server Model Diagram



The browser (client) sends a request to a server, and the server sends back a response — typically HTML, CSS, and JavaScript files that the browser then renders.

3. How a Browser Requests and Displays a Web Page

- **URL entry** — the user types a URL or clicks a link.
- **DNS lookup** — the browser asks a DNS server to translate the domain name into an IP address.
- **TCP/TLS connection** — the browser opens a connection to the server (HTTPS includes a TLS handshake for encryption).
- **HTTP request** — the browser sends an HTTP GET request asking for the page.
- **Server processing** — the server (possibly querying a database or running backend code) prepares a response.
- **HTTP response** — the server sends back HTML, along with links to CSS, JS, and image files.
- **Rendering** — the browser's rendering engine parses the HTML into a DOM, applies CSS, executes JavaScript, and paints the final page on screen.

4. Tools Required for a Web Development Environment

Tool	Purpose
Code editor (VS Code)	Writing and editing HTML/CSS/JS/backend code with syntax highlighting and extensions
Web browser (Chrome/Firefox)	Viewing and testing the site, using DevTools to debug
Version control (Git) + GitHub	Tracking changes to code and collaborating with others
Node.js & npm	Running JavaScript outside the browser, installing packages/libraries
Package manager (npm/yarn)	Managing project dependencies
Local/dev server (Live Server, XAMPP)	Previewing the site as it would run on a real server
Terminal / command line	Running build tools, scripts, and Git commands
Browser DevTools	Inspecting HTML/CSS, debugging JavaScript, checking network requests
Database tool (MySQL Workbench, MongoDB Compass)	Managing and viewing database data
API testing tool (Postman)	Testing backend API endpoints

5. What Is a Web Server?

A **web server** is software (often running on dedicated hardware) that stores website files and responds to browser requests by sending back web pages, typically over HTTP/HTTPS.

Commonly used web servers

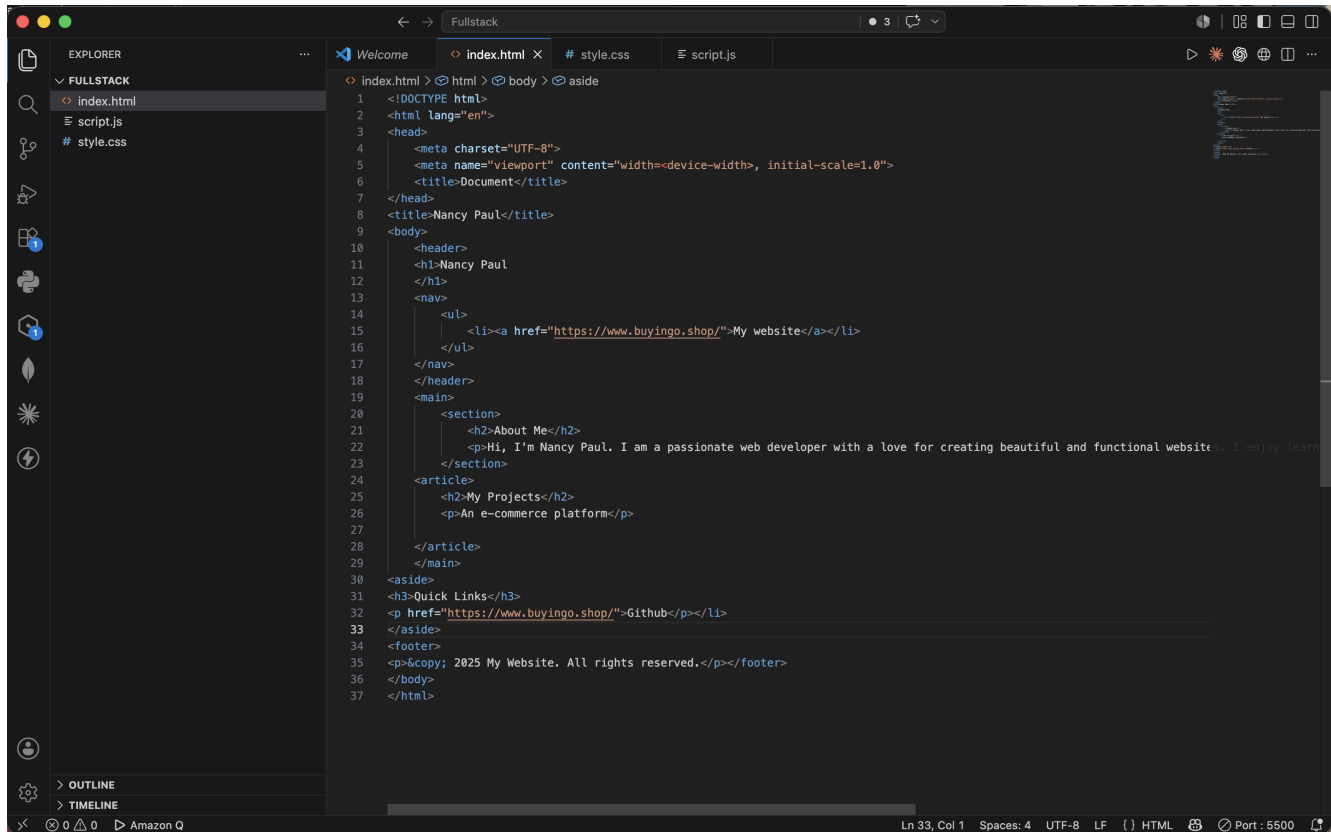
- **Apache HTTP Server** — one of the oldest and most widely used, highly configurable.
- **Nginx** — known for speed and handling many simultaneous connections; also used as a reverse proxy/load balancer.
- **Microsoft IIS** — used for Windows-based hosting, especially with .NET apps.
- **LiteSpeed** — a high-performance alternative to Apache, often used with WordPress hosting.
- **Node.js (with Express)** — not a traditional server but commonly used to build custom web servers in JavaScript.

6. Roles in a Web Project

- **Frontend developer**: builds the user interface, ensures the site is responsive and visually functional, connects the UI to backend APIs, and focuses on usability and performance in the browser.
- **Backend developer**: designs and builds server-side logic, APIs, authentication, and business rules; ensures the app can handle data securely and efficiently; connects the app to the database.
- **Database administrator (DBA)**: designs the database structure, manages data storage and backups, optimizes queries for performance, and maintains data security and integrity.

7. Installing and Configuring VS Code (with Screenshot)

VS Code was installed and configured with extensions for HTML, CSS, and JavaScript development (including Live Server for live preview). The screenshot below shows the working setup with a project open (index.html, style.css, script.js) and Live Server running on port 5500.



Setup steps

- Downloaded VS Code from code.visualstudio.com for the operating system in use.
- Ran the installer and enabled 'Add to PATH' and 'Open with Code' context menu options.
- Installed extensions: Live Server, Prettier, and HTML CSS Support.
- Created a project folder with index.html, style.css, and script.js.
- Used 'Open with Live Server' to preview the page in the browser with auto-refresh on save.

8. Static vs Dynamic Websites

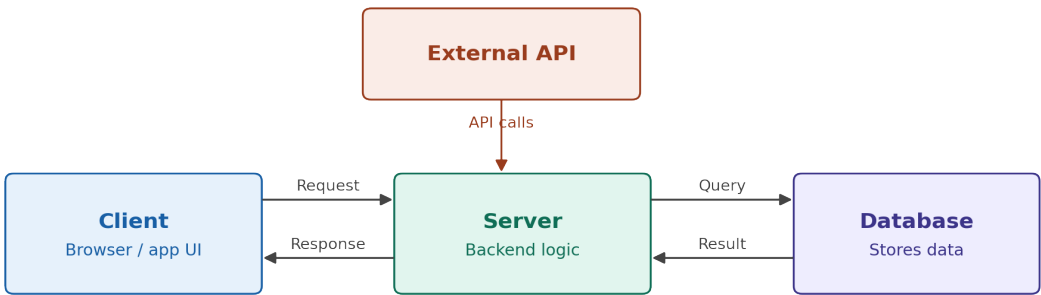
- **Static website:** content is fixed and the same for every visitor; pages are plain HTML/CSS files served as-is, with no server-side processing or database. Example: a simple personal portfolio page listing a resume and projects.
- **Dynamic website:** content changes based on user interaction, data, or logic run on the server (often pulling from a database). Example: an e-commerce site like Flipkart, where product listings, prices, and recommendations change per user and update in real time.

9. Five Web Browsers and Their Rendering Engines

Browser	Rendering Engine
Google Chrome	Blink (forked from WebKit)
Mozilla Firefox	Gecko
Apple Safari	WebKit
Microsoft Edge	Blink (switched from EdgeHTML in 2020)
Opera	Blink

Rendering engines are the part of the browser that parses HTML/CSS and paints the page. Because Chrome, Edge, and Opera all use **Blink**, they tend to render pages very similarly. **Gecko** (Firefox) and **WebKit** (Safari) are separate engines with their own quirks — occasionally a CSS feature or JavaScript API behaves slightly differently across them, which is why developers test across multiple browsers rather than assuming Chrome behavior applies everywhere.

10. Web Architecture Flow — Client, Server, Database, APIs



The **client** sends a request to the **server**; the server processes it — running logic, and possibly calling an **external API** for extra data (e.g. payments or maps) — and queries the **database** for stored data. The database returns results to the server, which formats a response and sends it back to the client to display.